

TinyTorch Reflections — Module 2

Activation Functions: ML Systems Reflection Questions

Answers use the same structured format as before, so you can keep building the habit of reasoning through mechanism rather than memorizing conclusions.

Use this structure for every question:

Concept → **Why** → **Example** → **Rule**

1. Computational Cost Comparison

ReLU is the most popular activation function in hidden layers. Why is ReLU computationally cheaper than Sigmoid or GELU?

Consider:

- What mathematical operations does ReLU require? (hint: just $\max(0, x)$)
- What operations does Sigmoid require? (hint: exponentials)
- For a hidden layer with 1 million neurons, how many $\exp()$ calls does each activation require?

Concept	ReLU vs. Sigmoid/GELU computational cost
Why	ReLU needs only a comparison and a max operation, while Sigmoid and GELU require an exponential, which is far more expensive to compute.
Example	For 1 million neurons, ReLU performs 1 million $\max(0, x)$ ops; Sigmoid performs 1 million $\exp()$ calls, each costing many more CPU/GPU cycles than a compare.
Rule	Prefer computationally simple activations like ReLU in hidden layers to minimize per-element cost at scale.

Key Insight: In production models with billions of parameters, small per-element costs compound. ReLU's simplicity makes it roughly 3–4x faster than Sigmoid.

2. Numerical Stability

In your Softmax implementation, why did we subtract the maximum value before computing exponentials?

Consider:

- What happens when you compute $\exp(1000)$?
- What about $\exp(1000) / (\exp(1000) + \exp(1001))$?
- Does subtracting a constant from all inputs change the final softmax output?

Concept	Max-subtraction trick in Softmax
Why	$\exp()$ overflows for large inputs, producing inf and then NaN; subtracting the max keeps every exponent argument at or below zero.

Example	$\exp(1000)$ overflows, but $\exp(1000 - \max) / \sum(\exp(x - \max))$ is numerically stable and mathematically identical to the unshifted version.
Rule	Always shift inputs by their max value before exponentiating to guarantee numerical stability without changing the result.

Key Insight: $\exp(x - \max) / \sum(\exp(x - \max)) = \exp(x) / \sum(\exp(x))$, because the constant factor cancels in the ratio — stability is free.

3. Sparsity and Efficiency

ReLU creates "sparsity" by zeroing negative values. Why might having many zero activations be beneficial for computation?

Consider:

- Memory: Do zeros need to be stored differently than non-zeros?
- Computation: What happens when you multiply by zero?
- Learning: If 50% of neurons are "off" for a given input, what does that mean for the representation?

Concept	Sparsity introduced by zeroing negative values
Why	Zero-valued activations contribute nothing to a multiply-accumulate, so sparsity-aware systems can skip that work and storage entirely.
Example	If 50% of neurons are zeroed for an input, sparse hardware/software can skip those multiply-adds, roughly halving compute for that layer.
Rule	Sparsity can reduce compute and memory when the system exploits it, though standard dense GPU kernels don't benefit automatically.

Key Insight: Sparse matrix formats and sparse-aware kernels are how frameworks turn ReLU's zeros into real memory and compute savings, rather than an automatic benefit.

4. Activation Selection for Different Stages

Why do we typically use different activations for hidden stages vs. output stages of a computation?

Consider the requirements:

- Hidden stages: need to preserve information flow, be efficient, and add nonlinearity.
- Binary classification output: needs values in (0, 1) representing a probability.
- Multi-class classification output: needs a probability distribution (values sum to 1).

Concept	Matching activation choice to functional requirement
Why	Hidden layers need cheap computation and stable gradients; output layers need values shaped to match what they represent.
Example	ReLU is used in hidden layers for speed and gradient flow; Sigmoid maps a single output to (0,1) for binary probability; Softmax normalizes outputs to sum to 1 for a multi-class distribution.

Rule	Choose hidden activations for computational efficiency and gradient flow, and output activations to match the required output distribution.
-------------	---

Key Insight: The activation at each stage exists to satisfy a specific constraint — nonlinearity and efficiency inside the network, calibrated output range at the end.

5. The "Dying ReLU" Problem

If ReLU outputs 0 for some inputs, those inputs get no signal passed through — they effectively "die." What situations might cause this, and why is it a problem?

Consider:

- If inputs to a ReLU are always negative, its output is permanently zero.
- Once a ReLU "dies," no information flows through it — it contributes nothing to the computation.
- A dead ReLU is wasted capacity: it takes up memory and compute but produces no useful signal.

Concept	Permanently inactive (dead) ReLU neurons
Why	If a neuron's pre-activation input is always negative — from poor initialization or a large gradient update — both its output and its gradient stay at zero forever, so it stops learning.
Example	A neuron that consistently receives negative input outputs 0, passes no signal forward, and receives no gradient to update itself.
Rule	Use ReLU variants such as LeakyReLU, PReLU, or GELU to preserve a small gradient for negative inputs and prevent neurons from dying permanently.

Key Insight: LeakyReLU ($f(x) = \max(0.01x, x)$), PReLU (learnable negative slope), and GELU (smooth, never fully zero) are the standard fixes used in practice.

Bonus Challenge: Memory Analysis

You're running inference on a model with a hidden layer of size (batch=32, features=4096) using different activations.

Calculate for each activation:

- How many bytes of output memory are needed? (assume float32)
- How many temporary buffers does Softmax need vs. ReLU?
- If you switch from float32 to float16, what's the memory savings?

Concept	Activation function memory footprint
Why	Activations produce an elementwise output the same shape as their input; only Softmax needs a small extra reduction buffer.

Example

(32, 4096) float32 output = $32 \times 4096 \times 4$ bytes = 524,288 bytes ($\approx$ 512 KB). ReLU needs no extra buffer; Softmax needs one scalar per row (32 floats, negligible) for the max/sum reduction. Switching to float16 halves the output to $\approx$ 256 KB.

Rule

Activation memory scales with output size (batch $\times$ features $\times$ dtype bytes), not with activation complexity; lower precision halves memory usage.

Key Insight: Activation functions are memory-light (output matches input size), but the choice of activation and precision still drives computational speed and numerical accuracy significantly.

Keep using this format throughout TinyTorch: short, technical, and focused on understanding the underlying system behavior.